Санкт-Петербургский политехнический университет Петра Великого

Отчёт по лабораторной работе № 3

Проектирование схемы базы данных с помощью языковой модели

По дисциплине «Автоматизация научных исследований»

Выполнила: Чинь Тхи Тху Хоай
Группа: 5040102/60201

Санкт-Петербург
2026

Содержание

1 Материалы и ход работы **2**
1.1 Полное исходное условие 2
1.2 Методика оценки 3

2 Первоначальная схема и проверка DBML **3**
2.1 Первый запрос и результат 3
2.2 Проверка синтаксиса 3

3 Качественный анализ первой версии **4**

4 Исправление схемы и повторная проверка **5**
4.1 Ответ модели на замечания 5
4.2 Проверочные случаи 6

5 Реализуемость операций и отчётов **6**
5.1 Соответствие требованиям задания 6
5.2 Оставшиеся ограничения 6

6 Заключение **8**

Диаграммы: до и после исправлений **9**

A Первоначальный запрос P0 **10**

B Запрос с замечаниями P1 **10**

1. Материалы и ход работы

Цель работы — оценить, насколько полно большая языковая модель преобразует текстовое описание задачи в реляционную схему базы данных, выявить проблемы первоначального решения и проанализировать исправление по содержательным замечаниям.

Работа выполнена по варианту № 13 «Тестирование студентов». Использована модель ChatGPT-6 в веб-интерфейсе. Результаты представлены в формате DBML и визуализированы в https://dbdiagram.io/. В качестве исходных артефактов сохранены `prompt1.txt`, `prompt2.txt`, ответы модели `ChatGPT-first-answer.txt`, `ChatGPT-second-answer.txt` и изображения диаграмм.

1.1. Полное исходное условие

13. Тестирование студентов

Есть перечень специальностей и списки предметов по каждой специальности (одни и те же предметы могут быть на разных специальностях). Есть списки тем по каждому предмету и вопросов по каждой теме (некоторые вопросы могут относиться к разным темам/предметам). Есть списки групп студентов по каждой специальности.

Требуется:

- Поддержка генерации набора тестов для студентов выбранной группы по заданной теме/темам с возможностью сделать одинаковый набор вопросов для всех или уникальные наборы вопросов для каждого студента группы;
- Поддержка внесения результатов тестирования с автоматической оценкой (неправильный ответ: -1 балл; неполный: $-0{,}5$; просрочка по времени со сдачей теста: $-0{,}5$);
- Поддержка архива результатов тестирования студентов;
- Поддержка внесения/изменения/удаления предметов, тем и вопросов тестирования.

Отчёты:

- Результаты конкретного теста для группы;
- Результаты тестирования группы/групп за период; список двоечников;
- Самые сложные вопросы.

Полный текст первоначально отправленного запроса с условием приведён в приложении А.

1.2. Методика оценки

Схема оценивалась в два этапа. На первом этапе проверялась возможность построения диаграммы в dbdiagram.io без ошибок и предупреждений. На втором этапе DBML сопоставлялся с условием по пяти критериям целостности: уникальность исходного вопроса в варианте, соответствие заданий специальности группы, неизменность выданных тестов, согласованность баллов и дат, уникальность индивидуальных вариантов.

Замечание считалось *устранённым*, если правило выражено действующим ограничением DBML (PK, UNIQUE, FK, CHECK); *частично устранённым* — если в DBML выражена только часть правила; *описанным* — если правило сформулировано в комментариях как требование к транзакции, триггеру или приложению и самой схемой не обеспечено. Запуск SQL-запросов и проверка триггеров на реальной СУБД не выполнялись.

2. Первоначальная схема и проверка DBML

2.1. Первый запрос и результат

В первом запросе модель должна была вернуть один полный блок DBML: таблицы, атрибуты, ключи и связи, достаточные для операций и отчётов. SQL-запросы и программный код не требовались. В ответе получена схема из **18 таблиц**, **2 перечислений** и **23 объявлений Ref**. Группы таблиц приведены в табл. 1.

2.2. Проверка синтаксиса

Схема была вставлена в dbdiagram.io и построена. Исходная версия не вызвала ошибок и предупреждений редактора. Первоначальная диаграмма представлена на рис. 1. Отсутствие предупреждений означает успешную проверку синтаксиса, но не гарантирует выполнения всех правил предметной области. Исходный ответ сохранён без изменений в `ChatGPT-first-answer.txt`

Таблица 1: Основные группы таблиц первоначальной схемы

Таблицы	**Назначение**
specialties, subjects, specialty_subjects	Связь специальностей с дисциплинами
topics, questions, topic_questions	Темы и вопросы, принадлежащие нескольким темам
question_revisions, question_options	Версии вопросов и варианты ответов
student_groups, students	Учебные группы и студенты
tests, test_topics, test_variants, test_variant_questions	Тесты, темы, варианты и состав вопросов
test_grade_thresholds	Пороговые значения оценок
test_attempts, test_answers, test_answer_options	Выдача заданий, ответы и результаты

3. Качественный анализ первой версии

Проверка проводилась по проверочным сценариям, в которых корректно построенная диаграмма может не обеспечивать требуемую целостность. Выявленные проблемы перечислены в табл. 2. В комментариях первой версии часть правил уже была названа (например, различие наборов вопросов в режиме individual), однако ни одно из них не было обеспечено ограничением схемы.

Таблица 2: Проблемы и последствия первоначальной схемы

Проблема и место	**Проверочный сценарий**	**Последствие и необходимое изменение**
test_variant_questions: ключ по редакции	В один вариант попадают две редакции одного исходного вопроса.	Одинаковый вопрос повторяется; нужно контролировать уникальность question_id в рамках варианта.
Связь выбранных тем со специальностью	Для группы формируется тест по дисциплине, не входящей в её специальность.	FK гарантируют существование записей, но не эту цепочку; нужна проверка при выдаче.

Проблема и место	**Проверочный сценарий**	**Последствие и необходимое изменение**
Изменяемость выданных тестов	После выполнения теста изменяются правильные ответы или назначенный вариант.	История и проверка ответов могут исказиться; нужны фиксированные редакции и запрет изменений.
Контроль дат и баллов	У несданной попытки проставлена оценка или итоговый балл отрицателен.	Состояние записей противоречиво; нужны CHECK и атомарная запись результата.
`individual`: уникальность комплектов	Разным студентам назначены варианты с разными номерами, но с одинаковыми вопросами.	Формально номера разные, а условие задачи нарушено; нужна проверка состава вопросов независимо от порядка и редакций.

Эти проблемы не выдаются синтаксическим анализатором редактора. Они получены сопоставлением DBML с полным условием задания. Замечания переданы в том же диалоге модели с требованием сохранить удачные решения первой версии (приложение Б).

4. Исправление схемы и повторная проверка

4.1. Ответ модели на замечания

На второй запрос ChatGPT вернул схему с теми же **18 таблицами**, **2 перечислениями** и **23 объявлениями Ref**, но с дополнительными проверками. В исправленном DBML появились **9 именованных CHECK** (по три в `tests`, `test_grade_thresholds` и `test_attempts`), а для связки вопроса и редакции добавлены составной внешний ключ и два уникальных индекса: `(question_id, id)` в `question_revisions` и `(test_id, variant_number, question_id)` в `test_variant_questions`. Фрагмент второй версии:

```
Table test_variant_questions {
  test_id bigint [not null]
  variant_number int [not null]
  question_id bigint [not null]
  question_revision_id bigint [not null]
  position int [not null]
  indexes {
    (test_id, variant_number, question_revision_id) [pk]
```

```
    (test_id, variant_number, question_id) [unique]
    (test_id, variant_number, position) [unique]
  }
}
Ref: test_variant_questions.(question_id, question_revision_id)
  > question_revisions.(question_id, id)
```

Изменения в двух таблицах наглядно показаны на рис. **??**.

Таблица 3: Результат исправления пяти замечаний

Замечание	**Изменение во второй версии**	**Статус**
Повтор редакций одного вопроса	Добавлены `question_id`, UNIQUE и составной FK	Устранено в DBML
Соответствие специальности	Проверка группы, дисциплины, темы и вопроса перед выдачей	Описано (транзакция)
Неизменность истории	Зафиксированы запреты правок выданных вариантов, редакций и результатов	Описано (триггер/приложение)
Даты и оценки	Девять CHECK плюс атомарный расчёт итогов, включая просрочку	Частично в DBML
Индивидуальные комплекты	Сравнение отсортированных множеств исходных `question_id`	Описано (транзакция)

Повторная вставка DBML позволила получить исправленную диаграмму (рис. 2). В предоставленном снимке видна построенная схема, однако панель проверки редактора не показана; поэтому **число предупреждений второй версии отдельно не подтверждено** [Заполнить: добавить снимок панели проверки dbdiagram.io для второй версии]. Исправленный код сохранён в `ChatGPT-second-answer.txt`.

4.2. Проверочные случаи

Эти результаты отражают *анализ конструкции*, а не запуск SQL-запросов или проверку триггеров на реальной базе данных.

5. Реализуемость операций и отчётов

5.1. Соответствие требованиям задания

5.2. Оставшиеся ограничения

Вторая версия уточняет правила, которые нельзя гарантировать обычными ключами и CHECK: допустимость темы для специальности, соответ-

Таблица 4: Проверка сценариев на уровне проектирования

Сценарий	**Результат анализа второй схемы**
Две редакции одного вопроса в варианте	UNIQUE по question_id не допускает такое дублирование
Назначен вопрос другой специальности	Правило транзакции отвергает выдачу; одного DBML недостаточно
Изменение ключа после выдачи	Описан запрет изменения используемой редакции; требуется реализация
Оценка до сдачи	CHECK согласует submitted_at, final_points, grade
Два индивидуальных варианта с одинаковым набором	Сравнение наборов описано, но не обеспечено UNIQUE
Студент переведён в другую группу	Историческая принадлежность для теста получается через tests.group_id

Таблица 5: Соответствие требованиям исходного задания

Операция или отчёт	**Данные и способ получения**
Создание одинаковых / разных тестов	tests, test_variants, test_variant_questions, test_attempts
Автоматическое оценивание	test_answers, question_options, test_attempts; расчёт по правилам транзакции
Архив результатов	tests.archived_at, test_attempts, сохранённые редакции вопросов
Изменение и удаление справочников	deleted_at и новые question_revisions, без удаления истории
Результаты выбранного теста	test_attempts по test_id с данными назначенных вариантов
Результаты за период, двоечники	Фильтрация по submitted_at; историческая группа из tests
Самые сложные вопросы	Доли partial и incorrect из test_answers, группировка по исходному вопросу

ствие группы назначенным студентам, уникальность наборов индивидуальных вариантов, запрет изменений после выдачи и полную проверку расчёта баллов. Они только описаны в комментариях и должны быть реализованы в транзакциях, триггерах или приложении.

Кроме замечаний, переданных модели, при чтении второго DBML выявлены дополнительные слабые места (проверено по тексту схемы, не на СУБД):

- среди 9 CHECK нет проверки `final_points` $\leq$ `initial_points`;
- требование «у каждой редакции вопроса есть хотя бы один правильный вариант» указано в комментарии, но не обеспечено;
- принадлежность студента группе из `tests.group_id` на момент выдачи в схеме не выражена;
- названия справочников и связи тем с вопросами берутся из актуального справочника, поэтому их последующее изменение может исказить интерпретацию исторических отчётов (упрощение отмечено самой моделью).

6. Заключение

В работе выполнено проектирование схемы базы данных по варианту № 13 «Тестирование студентов» с помощью ChatGPT. Первоначальная схема из 18 таблиц успешно визуализировалась в dbdiagram.io, но качественная проверка выявила пять проблем целостности данных.

После передачи замечаний в том же диалоге модель сохранила состав сущностей и число связей. Из пяти замечаний ограничениями DBML полностью устранено одно (повтор исходного вопроса в варианте), частично — одно (даты и оценки: число именованных CHECK выросло с 0 до 9); остальные три сформулированы как требования к транзакциям и триггерам и схемой не обеспечены. Дополнительно найдены не затронутые моделью слабые места (разд. 6).

Отсутствие ошибок DBML не гарантирует соответствия предметной области: содержательная проверка и уточняющий запрос улучшили схему, но основные межтабличные правила остаются требованиями к реализации.

Диаграммы: до и после исправлений

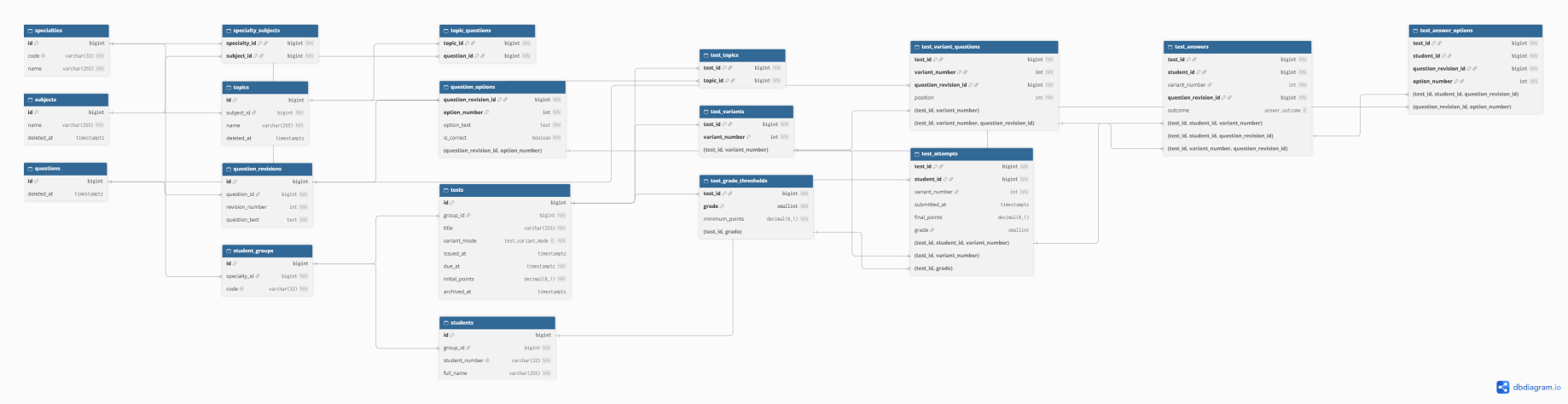


Рис. 1: Первоначальная схема базы данных (P0)

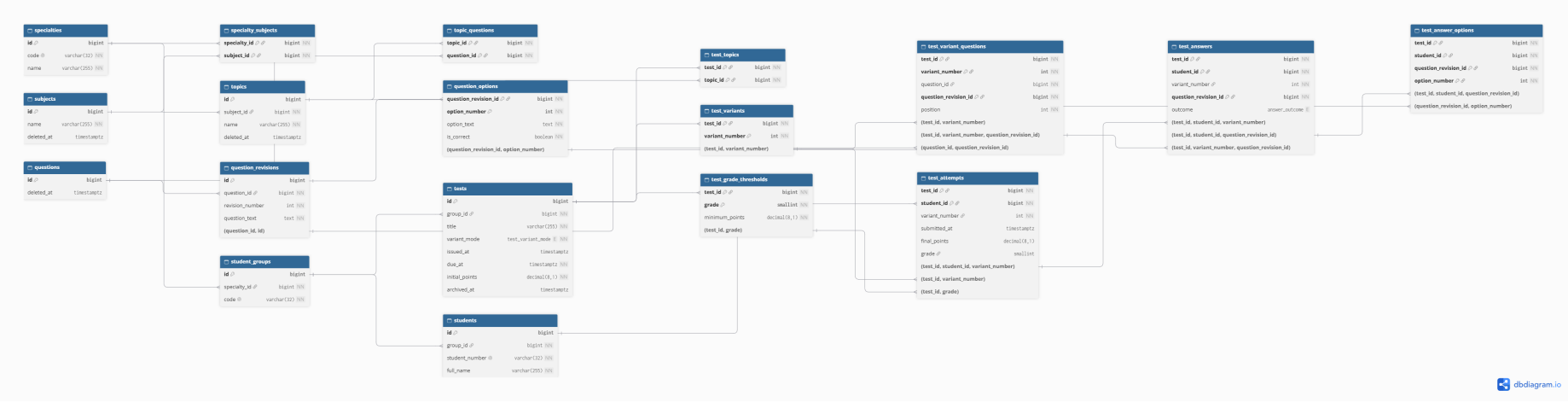


Рис. 2: Исправленная схема базы данных (P1)

A. Первоначальный запрос P0

```
По приведённому ниже заданию спроектируй реляционную схему базы данных.

Выдай один полный блок DBML для вставки в https://dbdiagram.io/.
Нужна только схема: таблицы, поля, типы данных, первичные и внешние ключи, связи.
Все операции и отчёты из условия должны быть реализуемы на её основе.
SQL-запросы и код приложения не нужны.
Если необходимы допущения, явно укажи их в комментариях DBML.

Полное условие задания:

13. Тестирование студентов

Есть перечень специальностей и списки предметов по каждой специальности (одни и те же предметы
↪  могут быть на разных специальностях). Есть списки тем по каждому предмету и вопросов по
↪  каждой теме (некоторые вопросы могут относиться к разным темам/предметам). Есть списки
↪  групп студентов по каждой специальности.

Требуется:
- Поддержка генерации набора тестов для студентов выбранной группы по заданной теме/темам с
↪  возможностью сделать одинаковый набор вопросов для всех или уникальные наборы вопросов для
↪  каждого студента группы;
- Поддержка внесения результатов тестирования с автом. оценкой (неправ.ответ: -1 балл;
↪  неполный: -0,5; просрочка по времени со сдачей теста: -0,5);
- Поддержка архива результатов тестирования студентов;
- Поддержка внесения/изменения/удаления предметов, тем и вопросов тестирования;

Отчеты:
- Результаты конкр. теста для группы;
- Результаты тестирования группы/групп за период; список двоечников;
- Самые сложные вопросы.
```

B. Запрос с замечаниями P1

```
Проверь и исправь ранее предложенную схему базы данных «Тестирование студентов». Это
↪  продолжение работы в том же диалоге. Сохрани все требования исходного задания и удачные
↪  решения первой версии.

Исходный DBML успешно построился в dbdiagram.io без ошибок и предупреждений. Однако
↪  качественная проверка выявила следующие проблемы.

1. Повторное включение одного вопроса в вариант.
В таблице test_variant_questions уникальность обеспечивается по question_revision_id. Разные
↪  редакции одного question_id могут попасть в один вариант теста. Это нарушает требование,
↪  что один исходный вопрос встречается в варианте не более одного раза. Предложи ограничение,
↪  сохраняющее возможность выбора конкретной редакции вопроса.

2. Соответствие заданий специальности.
Не гарантировано, что темы теста и вопросы относятся к предметам, доступным специальности
↪  выбранной группы. Предусмотри проверку цепочки группа — специальность — предмет — тема —
↪  вопрос. Если это невозможно выразить обычными FK или CHECK, явно укажи необходимую
↪  транзакционную проверку или бизнес-логику.

3. Неизменность выданных тестов и исторических данных.
После выдачи нельзя незаметно изменять набор вопросов, редакции, варианты ответов, ключи
↪  правильных ответов, параметры оценивания и назначенные варианты. Сохрани историю
↪  результатов и возможность редактирования справочника для будущих тестов. Раздели реально
↪  действующие ограничения DBML и правила, требующие триггеров или бизнес-логики.
```

4. Согласованность оценок и дат.
Добавь допустимые ограничения для неотрицательных баллов, корректного срока сдачи, диапазона оценок 2–5 и согласованности submitted_at, final_points, grade. Проверки агрегированного результата по ответам и штрафам не выдавай за обычные CHECK. Уточни, как однократно учитывается штраф за просрочку и как сохраняются результаты автоматической проверки.

5. Уникальность индивидуальных вариантов.
В режиме individual номера вариантов различаются, но состав вопросов может совпадать. Предусмотри способ проверки уникальности наборов исходных question_id независимо от порядка вопросов и редакций. Для режима common всем студентам должен назначаться один общий вариант. Если эти правила требуют бизнес-логики, опиши её явно.

Дополнительно проверь:
- сохранение истории после перевода студента в другую группу;
- допустимость назначения теста только студентам нужной группы на момент выдачи;
- корректность составных внешних ключей;
- возможность получить все отчёты из исходного задания;
- отсутствие ненужного дублирования данных.

Не удаляй полезные таблицы и связи без обоснования. Не добавляй новые сущности, если проблему можно решить ограничениями существующей схемы.

Верни один полный исправленный DBML для dbdiagram.io, а не отдельные фрагменты изменений. В комментариях DBML укажи сделанные исправления, необходимые допущения и ограничения, которые должны обеспечиваться приложением или триггерами.

SQL-запросы и код приложения не нужны. Не утверждай, что схема проверена в dbdiagram.io, если фактически не выполнял эту проверку.